

AI 协作实践

AI 时代，如何与模型和工具长期协作

个人实践心得 · 可复制的思路与节奏

作者 吕韧 日期 2026 年 5 月

目 录

开 场

01 我们面对的现实

核心理念

02 最佳实践会过期，底层逻辑不会

03 先规划，再交给 AI

策略层

04 贵模型想明白，普通模型干明白

05 Cursor / Codex：从写代码到做「严肃脑力活」

方法层

06 复杂项目：试试「Sprint 节奏」

07 让代码和测试成为「活文档」

08 AI 编程仍然绕不开「真工程」

沉淀层

09 每一次踩坑，都该变成下一项目的垫脚石

10 习惯 Markdown，降低协作摩擦

结 语

11 三条带走就能用的原则

开 场

开场

我们面对的现实

- 模型与工具几乎「按月换代」，照搬旧教程容易失效
- 真正稳定的是：**底层逻辑 + 试错 + 复盘**

本分享目标：给你一套可迁移的协作框架，而不是工具清单

核心理念

核心理念

最佳实践会过期，底层逻辑不会

- 模型进化太快 → 不要迷信「唯一正确流程」
- 有效路径：优秀底层逻辑 + 不断尝试 + 总结反思
- 持续探索模型/工具进化，并迭代自己的工作方式

核心理念

先规划，再交给 AI

懒人模式效果最差

- **人负责**：目标、约束、优先级、验收标准
- **AI 负责**：实现、扩写、重构、加速
- 充分思考与规划是 ROI 最高的投入

策略层

策略层 贵模型想明白，普通模型干明白

场景	建议模型档位
架构规划、方案权衡、疑难排障	高能力模型
批量修改、模板代码、重复劳动	普通模型

原则：质量与成本主动分层，避免全程顶配

策略层 Cursor / Codex：从写代码到做「严肃脑力活」

- 适用：复杂调研、规划、方案设计
- 优势：文档化缓解上下文限制
- 对比网页版：投喂资料与沉淀产出都更可持续

方法层

方法层 复杂项目：试试「Sprint 节奏」

- 每个 Sprint：规划 → 执行 → 验收 → 总结
- 可能更慢，但清晰度、可控性显著提升
- 适合：需求多变、周期长、多轮 AI/多人接手的项目

方法层 让代码和测试成为「活文档」

- **实例化需求**：用具体用例描述行为，减少歧义
- **TDD**：先定义可验证行为，再驱动实现
- 配合 Rules / Skills 固化规范

结果：质量 ↑、可维护性 ↑、人机交接成本 ↓

方法层 AI 编程仍然绕不开「真工程」

- 版本管理、CI/CD、部署、环境——问题密集且琐碎
- 非软件背景用户易在此受挫 → 也是是否适合 AI 编程的试金石
- **对策**：让 AI 把经验文档化（注意事项、步骤、排障记录）

沉淀层

沉淀层

每一次踩坑，都该变成下一项目的垫脚石

- **项目内：**需求说明、架构决策、部署 Runbook、复盘
- **项目间：**可复用的检查清单与模板
- 对新 AI / 新同事：文档即上下文，降低重复解释成本

沉淀层

习惯 Markdown，降低协作摩擦

- 结构清晰、版本友好、AI 易读易改
- 推荐工具：[MarkText](#) (开源 Markdown 编辑器)
- 实践建议：需求、方案、会议纪要、运维手册统一使用 .md

Markdown 是人与 AI 协作最通用的文档格式

三条带走就能用的原则

1 人先想清楚，再让 AI 大规模动手

2 复杂项目：Sprint 节奏 + 实例化需求 + TDD

3 一切经验文档化，用 Markdown 连接人与 AI

AI 不会替代思考；它放大的是——你已经想明白的那部分。

单页话术参考

页	一句话版本
03	工具会变，拆解问题、验证结果、复盘迭代不会变。
04	AI 是加速器，不是方向盘；方向盘一直在你手里。
05	规划用贵的，搬砖用省的，别全程烧顶配。
06	把 Cursor 当「带仓库的 AI 工作台」，不只当补全工具。
07	和 AI 协作也要里程碑，否则只会无限对话。
08	测试用例就是需求说明书，代码是实现附录。
09	能搞定 Git、CI、部署，你才真的在「做软件」而不只是「生成代码」。
10	文档是给未来的你和未来的 AI 写的。